10.2 Assignment: Recommender System

Ryan Maxam
DSC 630
August 16, 2026

Objective

The main goal of this week's assignment is to build a recommender system using the provided small MovieLens dataset. By creating this recommender system, a user can enter a movie type, and the system will return 10 other movies with similar rating patterns among MovieLens users.

1. Dataset and Method

I am using the MovieLens Latest small dataset provided by the GroupLens Research Project. I have found that it contains 100,836 ratings, 3,683 tag applications, 9,742 movies, and 610 users. The ratings are based on a star scale from 0.5 to 5.0.

My recommender uses item-based collaborative filtering. By using this, collaborative filtering learns from users' behaviors, rather than recommending movies only because of shared genres.

We are using a mean-centered rating to reduce the effect of users who rate everything high or low by subtracting each user's average rating from their individual rating. The movies will then be represented as vectors across users, and cosine similarity is used to measure how similar those vectors are.

I am also including a popularity safeguard that requires users to have at least 5 ratings to help prevent a movie with only one or two ratings from appearing as a top recommendation.

2. Import Libraries

For this notebook, I am using pandas and NumPy for data handling, SciPy for a sparse movie user matric, and scikit learn for cosine similarity.

In [1]:

```python
# Import the libraries used in the recommender system
from pathlib import Path
import numpy as np
import pandas as pd
from scipy.sparse import csr_matrix
from sklearn.metrics.pairwise import cosine_similarity

# Display options make notebook output easier to read
pd.set_option("display.max_colwidth", 80)
pd.set_option("display.float_format", lambda x: f"{x:.3f}")
```

3. Load the MovieLens Files

In [2]:

```python
DATA_DIR = Path(".")
if not (DATA_DIR / "movies.xlsx").exists():
    DATA_DIR = Path("/mnt/data")

# The converted movies, tags, and links workbooks contain a worksheet-title
# row above the real headers, so header=1 is used for those three files.
movies = pd.read_excel(DATA_DIR / "movies.xlsx", header=1)
ratings = pd.read_excel(DATA_DIR / "ratings.xlsx")
tags = pd.read_excel(DATA_DIR / "tags.xlsx", header=1)
links = pd.read_excel(DATA_DIR / "links.xlsx", header=1)

print("movies.xlsx shape:", movies.shape)
print("ratings.xlsx shape:", ratings.shape)
print("tags.xlsx shape:", tags.shape)
print("links.xlsx shape:", links.shape)
```

```
movies.xlsx shape: (9742, 3)
ratings.xlsx shape: (100836, 4)
tags.xlsx shape: (3683, 4)
links.xlsx shape: (9742, 3)
```

4. Inspect the Data

The two files required for this model are `movies.xlsx` and `ratings.xlsx`. I converted these to excel formatted copies to easily upload into my notbook.

In [3]:

```python
# Preview the movie information
display(movies.head())

# Preview the user ratings
display(ratings.head())
```

	movieId	title	genres
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
1	2	Jumanji (1995)	Adventure|Children|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	5	Father of the Bride Part II (1995)	Comedy

	userId	movieId	rating	timestamp
0	1	1	4.000	964982703
1	1	3	4.000	964981247
2	1	6	4.000	964982224
3	1	47	5.000	964983815
4	1	50	5.000	964982931

In [4]:

```python
# Basic quality checks
print("Unique users:", ratings["userId"].nunique())
print("Unique rated movies:", ratings["movieId"].nunique())
print("Total ratings:", len(ratings))
print("Missing values in movies.xlsx:", int(movies.isna().sum().sum()))
print("Missing values in ratings.xlsx:", int(ratings.isna().sum().sum()))
print("Duplicate rating rows:", int(ratings.duplicated().sum()))
print("Minimum rating:", ratings["rating"].min())
print("Maximum rating:", ratings["rating"].max())
```

```
Unique users: 610
Unique rated movies: 9724
Total ratings: 100836
Missing values in movies.xlsx: 0
Missing values in ratings.xlsx: 0
Duplicate rating rows: 0
Minimum rating: 0.5
Maximum rating: 5.0
```

5. Prepare the Ratings

I wanted to reduce the user-level bias in this analysis, and we did this by having the code calculate each user's average rating and subtract it from every rating by that user.

For example, if a specific user has an average rating of 3.0 and rates a movie 5.0, the adjusted rating is +2.0. This helps center each user on zero and focus the comparison on relative preference.

In [5]:

```python
# Calculate each user's average rating
ratings_prepared = ratings.copy()
ratings_prepared["user_mean"] = ratings_prepared.groupby("userId")["rating"].transform("mean")

# Mean-center every rating to reduce individual rating-scale bias
ratings_prepared["adjusted_rating"] = (
    ratings_prepared["rating"] - ratings_prepared["user_mean"]
)

display(ratings_prepared.head())
```

	userId	movieId	rating	timestamp	user_mean	adjusted_rating
0	1	1	4.000	964982703	4.366	-0.366
1	1	3	4.000	964981247	4.366	-0.366
2	1	6	4.000	964982224	4.366	-0.366
3	1	47	5.000	964983815	4.366	0.634
4	1	50	5.000	964982931	4.366	0.634

6. Build a Sparse Movie-User Matrix

A SciPy CSR sparse matric is used to store only the observed values which makes this approach more memory efficient. Each row is a movie, each column is a user, and the store value is the users mean centered rating for the movie. If we have any missing ratings, those are marked as zero.

In [6]:

```python
# Create stable row and column mappings for movie IDs and user IDs
rated_movie_ids = np.sort(ratings_prepared["movieId"].unique())
user_ids = np.sort(ratings_prepared["userId"].unique())

movie_to_row = {movie_id: i for i, movie_id in enumerate(rated_movie_ids)}
user_to_col = {user_id: i for i, user_id in enumerate(user_ids)}

row_indices = ratings_prepared["movieId"].map(movie_to_row).to_numpy()
col_indices = ratings_prepared["userId"].map(user_to_col).to_numpy()
values = ratings_prepared["adjusted_rating"].to_numpy()

movie_user_matrix = csr_matrix(
    (values, (row_indices, col_indices)),
    shape=(len(rated_movie_ids), len(user_ids))
)

print("Movie-user matrix shape:", movie_user_matrix.shape)
print("Stored non-zero values:", movie_user_matrix.nnz)
```

```
Movie-user matrix shape: (9724, 610)
Stored non-zero values: 100836
```

7. Create Supporting Movie Statistics

For our recommendation output, it will include each canidate movie numbers of ratings and average rating. The rating count will also be used as a minimum support filter.

In [7]:

```python
# Precompute lookup tables used by the recommendation function
movie_title_lookup = movies.set_index("movieId")["title"].to_dict()
movie_genre_lookup = movies.set_index("movieId")["genres"].to_dict()
movie_rating_count = ratings.groupby("movieId").size()
movie_average_rating = ratings.groupby("movieId")["rating"].mean()
```

8. Recommendation Functions

The first helper searches for movies by title. The main `recommend_movies()` function then:

1. Finds the requested movie.
2. Gets the mean-centered user-rating vector.
3. Compares the vector with every rated movie using cosine similarity.
4. Removes the selected movie itself.
5. Filters out candidates with fewer than five ratings.
6. Sorts the remaining movies from most to least similar.
7. Returns the top ten recommendations.

In [8]:

```python
def find_movie_titles(search_text, max_results=15):
    """Return MovieLens titles containing the user's search text."""
    search_text = str(search_text).strip()
    matches = movies[
        movies["title"].str.contains(search_text, case=False, regex=False, na=False)
    ][["movieId", "title", "genres"]]
    return matches.head(max_results).reset_index(drop=True)


def recommend_movies(movie_title, n_recommendations=10, min_ratings=5):
    """
    Recommend movies with rating patterns similar to the entered movie.

    Parameters
    ----------
    movie_title : str
        Full or partial title of a movie in movies.xlsx.
    n_recommendations : int
        Number of recommendations to return. Default is 10.
    min_ratings : int
        Minimum number of ratings required for a candidate movie.

    Returns
    -------
    pandas.DataFrame
        Ranked recommendations with similarity, rating count, average rating,
        and genres.
    """
    movie_title = str(movie_title).strip()

    # First search for all titles containing the text entered by the user.
    matches = movies[
        movies["title"].str.contains(movie_title, case=False, regex=False, na=False)
    ].copy()

    if matches.empty:
        print(f'No movie containing "{movie_title}" was found in the dataset.')
        print("Try another title or use find_movie_titles('part of title').")
        return pd.DataFrame()

    # Prefer an exact title match when one exists.
    exact = matches[matches["title"].str.lower() == movie_title.lower()]
    if len(exact) == 1:
        selected = exact.iloc[0]
    elif len(matches) == 1:
        selected = matches.iloc[0]
    else:
        print("Multiple movies matched that search. Please enter a more specific title:")
        display(matches[["title", "genres"]].head(15).reset_index(drop=True))
        return pd.DataFrame()

    selected_movie_id = int(selected["movieId"])

    # Some titles may appear in movies.xlsx without a usable rating vector.
    if selected_movie_id not in movie_to_row:
        print(f'{selected["title"]} does not have enough rating data for this model.')
        return pd.DataFrame()

    # Calculate cosine similarity between the selected movie and all rated movies.
    selected_vector = movie_user_matrix[movie_to_row[selected_movie_id]]
    similarities = cosine_similarity(selected_vector, movie_user_matrix).ravel()

    # Build a result table with useful descriptive information.
    results = pd.DataFrame({
        "movieId": rated_movie_ids,
        "similarity": similarities
    })
    results["title"] = results["movieId"].map(movie_title_lookup)
    results["genres"] = results["movieId"].map(movie_genre_lookup)
    results["rating_count"] = results["movieId"].map(movie_rating_count).fillna(0).astype(int)
    results["average_rating"] = results["movieId"].map(movie_average_rating)

    # Exclude the selected movie and weakly supported/negative matches.
    results = results[
        (results["movieId"] != selected_movie_id) &
        (results["rating_count"] >= min_ratings) &
        (results["similarity"] > 0)
    ]

    # Rank primarily by cosine similarity, using rating count only as a tie-breaker.
    results = results.sort_values(
        ["similarity", "rating_count"], ascending=[False, False]
    ).head(n_recommendations).copy()

    results.insert(0, "rank", range(1, len(results) + 1))
    results["similarity"] = results["similarity"].round(3)
    results["average_rating"] = results["average_rating"].round(2)

    print(f'Recommendations based on: {selected["title"]}')
    return results[[
        "rank", "title", "genres", "similarity", "rating_count", "average_rating"
    ]].reset_index(drop=True)
```

9. User Input and Demonstration

In [9]:

```python
# USER INPUT CELL
# Replace the title below with any movie title from the MovieLens dataset.
favorite_movie = "Toy Story (1995)"

# Optional console prompt instead of the line above:
# favorite_movie = input("Enter a movie you like: ")

recommendations = recommend_movies(favorite_movie, n_recommendations=10)
display(recommendations)
```

```
Recommendations based on: Toy Story (1995)
```

	rank	title	genres	similarity	rating_count	average_rating
0	1	Toy Story 2 (1999)	Adventure|Animation|Children|Comedy|Fantasy	0.403	97	3.860
1	2	Toy Story 3 (2010)	Adventure|Animation|Children|Comedy|Fantasy|IMAX	0.327	55	4.110
2	3	Aladdin (1992)	Adventure|Animation|Children|Comedy|Musical	0.327	183	3.790
3	4	Wallace & Gromit: The Wrong Trousers (1993)	Animation|Children|Comedy|Crime	0.305	56	4.040
4	5	Back to the Future (1985)	Adventure|Comedy|Sci-Fi	0.277	171	4.040
5	6	Incredibles, The (2004)	Action|Adventure|Animation|Children|Comedy	0.275	125	3.840
6	7	Blazing Saddles (1974)	Comedy|Western	0.272	62	3.940
7	8	Finding Nemo (2003)	Adventure|Animation|Children|Comedy	0.263	141	3.960
8	9	Ghostbusters (a.k.a. Ghost Busters) (1984)	Action|Comedy|Sci-Fi	0.250	120	3.780
9	10	Raiders of the Lost Ark (Indiana Jones and the Raiders of the Lost Ark) (1981)	Action|Adventure	0.237	200	4.210

Example title search

If the exact MovieLens title is unknown, the helper below can be used to find matching titles before running the recommender.

In [10]:

```python
# Example: search for titles containing the words "Dark Knight"
display(find_movie_titles("Dark Knight"))
```

	movieId	title	genres
0	58559	Dark Knight, The (2008)	Action|Crime|Drama|IMAX
1	91529	Dark Knight Rises, The (2012)	Action|Adventure|Crime|IMAX
2	98124	Batman: The Dark Knight Returns, Part 1 (2012)	Action|Animation|Sci-Fi
3	99813	Batman: The Dark Knight Returns, Part 2 (2013)	Action|Animation

10. Interpretation of the Results

As you can see from the example we gave for Toy Story, the recommender placed movies such as Toy Story 2 and Toy Story 3 near the top of the recommendations because the patterns of ratings across MovieLens users are similar for the original Toy Story. We also saw that other family and adventure titles populated, even though genre labels were not specifically used to calculate similarity.

This is an important characteristic of collaborative filtering because the model can discover relationships based on user behavior rather than relying on manually defined movie attributes. I want to note that the similarity score should be used as a measure of aligned rating patterns, not as a predicted star rating or a guarantee that all viewers will enjoy the recommendations.

11. Limitations and Possible Improvements

We created this recommender for this week's assignment, and this can cause limitations such as:

- The dataset is sparse, meaning that less popular movies have less information than more popular ones.
- A single favorite movie provides less information about a user's interests than several ratings.
- We are only using rating behaviors for this assignment and not including genre, tags, cast, or directors.
- The MovieLens Small dataset is a development dataset and is not continuously updated.

If we wanted to create a more advanced version, it would combine the collaborative filtering with genres and tags, matrix factorization, and learn from several movies a user likes.

12. Conclusion

Overall, we have completed an analysis that allows a user to enter a movie from the MovieLens dataset and receive ten other movies to consider watching. The notebook loads smoothly, validates the data, mean-centers user ratings, creates a sparse movie-user matrix, computes item-to-item cosine similarity, includes a minimum-rating safeguard, and ranks the most similar movies.

The analysis is an example of item-based collaborative filtering because the recommendations are based on collective rating behavior. This approach is fully transparent, efficient for the dataset we are working with, and easy to reuse by changing just the movie title in the user input cell.

References

Harper, F. M., & Konstan, J. A. (2015). *The MovieLens Datasets: History and Context*. ACM Transactions on Interactive Intelligent Systems, 5(4), Article 19. https://doi.org/10.1145/2827872

GroupLens Research. (2018). *MovieLens Latest Small Dataset / README*. https://grouplens.org/datasets/movielens/latest/

Sarwar, B., Karypis, G., Konstan, J. A., & Riedl, J. (2001). *Item-based collaborative filtering recommendation algorithms*. Proceedings of the 10th International Conference on World Wide Web, 285-295. https://doi.org/10.1145/371920.372071

Scikit-learn Developers. *cosine_similarity - scikit-learn documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html